

DevOps Industrial Training

Project 2 – Version Control with Git and GitHub

Student Name: Salman Ahmad

Project: DevOps Project 2

Project Title: Version Control with Git

Training Program: DevOps Industrial Training

Batch: 2026

1. Introduction

This project was completed as part of my DevOps Industrial Training. The main purpose of the project was to understand how Git and GitHub are used to manage project files and maintain different versions of code.

During this project, I worked with a simple HTML and CSS website. I used Git to track the changes made to the project and GitHub to store and manage the project remotely.

I also worked with an AWS EC2 Ubuntu server and Nginx to understand how a website can be deployed on a Linux server.

2. Objectives

The main objectives of this project were:

- To understand the basic Git workflow.
- To work with a Git repository.
- To track changes using Git.
- To create and manage commits.
- To push project files to GitHub.
- To create and work with a separate Git branch.
- To understand Pull Requests.

- To merge changes into the main branch.
- To deploy an HTML/CSS website using Nginx.
- To understand the Nginx document root on an Ubuntu server.

3. Task 1 – Version Control with Git and GitHub

3.1 Project Setup

For this task, I worked with a simple responsive HTML and CSS website. The project contained the main HTML page, CSS stylesheet, and README file.

The project was cloned from a GitHub repository and then connected to my own GitHub repository.

The GitHub repository used for the project was:

Repository: <https://github.com/Salman-Ahmad542/DecodeLabs-project-2>

The main project files included:

index.html

style.css

README.md

3.2 Git Repository and Remote Repository

I used Git commands to check and manage the repository.

The remote repository was configured with my GitHub repository:

git remote -v

This allowed me to verify that the local project was connected to the correct GitHub repository.

3.3 Creating a Feature Branch

Instead of making all changes directly on the main branch, I created a separate feature branch.

The branch used was:

feature/update-page

The purpose of using a separate branch was to keep the new changes separate from the main project until they were ready to be merged.

3.4 Updating the Website

I updated the website design and content.

The homepage was improved with:

- A navigation bar
- DevOps project introduction
- Project features section
- Git and GitHub learning section
- About project section
- Git workflow section
- Footer section
- Responsive design for smaller screens

The website was styled using CSS to make the page cleaner and more professional.

3.5 Committing Changes

After making the changes, I used Git to track and commit the updated files.

The basic workflow was:

git status

git add .

git commit -m "Update project homepage"

The commit created a record of the changes made to the project.

3.6 Pushing Changes to GitHub

After committing the changes, I pushed the feature branch to GitHub.

The branch was:

feature/update-page

This allowed the changes to become available on the remote GitHub repository.

During the process, GitHub authentication was required. I used a GitHub Personal Access Token because GitHub no longer accepts a normal account password for Git operations over HTTPS.

3.7 Pull Request

After pushing the feature branch, I created a Pull Request on GitHub.

The Pull Request was created from:

feature/update-page

into:

main

The Pull Request title was:

Update project homepage

The purpose of the Pull Request was to review the changes before merging them into the main branch.

3.8 Merging the Pull Request

After creating the Pull Request, the changes were merged into the `main` branch.

After the merge, I switched back to the main branch and updated the local repository:

git checkout main

git pull origin main

I then checked the repository status:

git status

The final result showed:

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

This confirmed that the local main branch was synchronized with the GitHub repository.

4. Git Workflow Used

The overall Git workflow followed in this project was:

Project Files

↓

Git Repository

↓

Create Feature Branch

↓

Make Changes

↓

git add

↓

git commit

↓

git push

↓

Pull Request



Merge into main



git pull

This helped me understand how Git branches and GitHub Pull Requests can be used to manage project changes.

5. Task 2 – Deploying Website Using Nginx

5.1 Introduction

In the second part of the project, I worked with an AWS EC2 Ubuntu server and Nginx.

The purpose was to place the HTML and CSS files in the Nginx web server directory so that the website could be served by Nginx.

5.2 Nginx Document Root

The Nginx document root used for the website was:

`/usr/share/nginx/html/`

This is the directory where the website files were placed.

5.3 Website Files

The following files were copied to the Nginx document root:

`index.html`

`style.css`

I also verified the files using:

`ls -la /usr/share/nginx/html/`

The files were present in the directory with appropriate read permissions.

5.4 File Permissions

The website files were checked to make sure that Nginx could access them.

The expected permissions were:

```
sudo chmod 755 /usr/share/nginx/html
```

```
sudo chmod 644 /usr/share/nginx/html/index.html
```

```
sudo chmod 644 /usr/share/nginx/html/style.css
```

These permissions allow the Nginx web server to access and read the website files.

6. Nginx Deployment Result

After placing the website files in the Nginx document root, I tested the website through the EC2 server's public IP address.

However, instead of displaying the website, Nginx returned:

403 Forbidden

nginx/1.28.3 (Ubuntu)

The website files were present in the correct Nginx directory, so the issue appeared to be related to the Nginx configuration or access settings rather than missing website files.

The next troubleshooting steps were to check the Nginx configuration and validate it using:

```
nginx -t
```

and to inspect the default Nginx server configuration:

```
cat /etc/nginx/sites-enabled/default
```

Therefore, at the time of preparing this report, the Nginx deployment had reached the file-copy and configuration-troubleshooting stage, but the **403 Forbidden** issue had not yet been fully resolved.

7. Problems Faced During the Project

During the project, I faced several practical issues.

GitHub Authentication

GitHub did not accept a normal account password when pushing code. I solved this by using a GitHub Personal Access Token for authentication.

Branch and Pull Request Workflow

I learned how to create a feature branch, push changes to that branch, create a Pull Request, and merge it into the main branch.

Nginx 403 Forbidden Error

While deploying the website on Nginx, the files were successfully placed inside:

`/usr/share/nginx/html/`

but Nginx displayed a **403 Forbidden** error. This required further checking of the Nginx server configuration and permissions.

8. Skills Learned

Through this project, I gained practical experience with:

- Git
- GitHub
- Git branches
- Git commits
- Git push and pull

- Pull Requests
- Merging branches
- GitHub authentication using Personal Access Token
- Linux commands
- AWS EC2
- Nginx
- Website deployment
- File permissions
- Basic troubleshooting

9. Conclusion

This project helped me understand the basic workflow of version control using Git and GitHub.

I practiced managing project files, creating commits, working with a separate feature branch, pushing changes to GitHub, creating a Pull Request, and merging the changes into the main branch.

I also gained practical experience with deploying an HTML/CSS website on an AWS EC2 Ubuntu server using Nginx. Although the final deployment produced a 403 `Forbidden` error, the website files were successfully placed in the correct Nginx document root and the troubleshooting process helped me understand that server configuration and permissions need to be checked when a web server cannot serve the files.

Overall, the project gave me practical experience with basic Git/GitHub version control and Linux-based web deployment, which are important parts of a DevOps workflow.